PROJET : SYSTEME DE GESTION DES MISES A JOUR EN POWERSHELL

CONTEXTE :

Une entreprise dispose d'un parc de postes de travail sous Windows interconnectés au sein d'un réseau local. L'administration de ces machines est réalisée depuis un poste d'administration centralisé.

La sécurité du système d'information dépend notamment de la capacité de l'entreprise à maintenir régulièrement ses systèmes à jour. Lorsqu'une vulnérabilité est découverte dans Windows ou dans l'un de ses composants, Microsoft peut publier un correctif destiné à supprimer ou à réduire cette vulnérabilité. Un poste qui ne dispose pas des correctifs de sécurité nécessaires peut représenter un point faible dans le système d'information. Il peut être exploité par un attaquant et éventuellement servir de point de départ pour compromettre d'autres machines du réseau.

Le Patch Management désigne l'ensemble des activités permettant de gérer les correctifs logiciels au sein d'un parc informatique. Il comprend notamment :

- L'inventaire des postes ;
- L'identification des mises à jour installées ;
- **La détection des correctifs manquants ;**
- Le contrôle de conformité des machines ;
- **La centralisation des résultats ;**
- **La production de rapports ;**
- Le suivi des machines non conformes ou inaccessibles.

Dans un parc comportant plusieurs machines, effectuer ces contrôles manuellement devient rapidement inefficace. L'utilisation de PowerShell et de *PowerShellRemoting* permet d'automatiser ces opérations et d'administrer plusieurs postes à distance depuis une machine centrale.

Dans ce projet, un fichier nommé : `computers.txt` sera utilisé comme inventaire du parc. Il contiendra, pour chaque poste, son nom et son adresse IP, par exemple :

```
PC01;192.168.10.11
PC02;192.168.10.12
PC03;192.168.10.13
PC04;192.168.10.14
```

Le script devra utiliser ces informations pour identifier les machines à contrôler et pour réaliser les opérations d'administration distante.

La mise en œuvre du projet ne nécessite pas obligatoirement de disposer d'un grand nombre de machines virtuelles Windows. Vous pouvez utiliser une infrastructure réduite composée de :

- La machine Windows hôte comme poste d'administration ou comme poste supervisé;
- Une ou deux machines virtuelles Windows supplémentaires ;

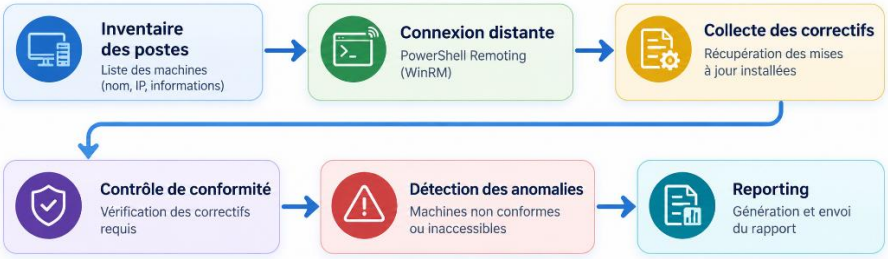
OBJECTIF DU PROJET

L'objectif de ce projet est de concevoir une solution simplifiée de Patch Management Windows à l'aide de PowerShell.

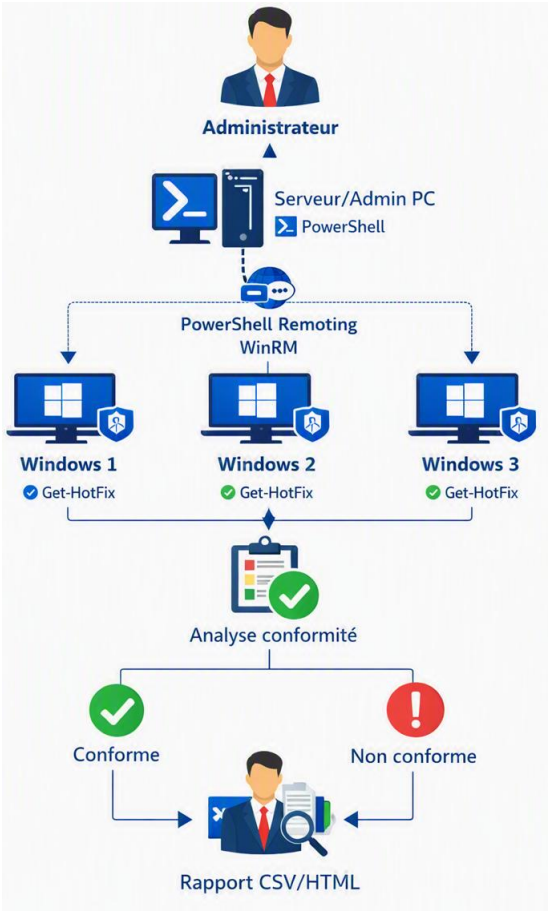
Vous devrez développer un ou plusieurs scripts permettant d'exploiter le fichier `computers.txt` afin de contrôler automatiquement l'état de plusieurs postes Windows.

La solution devra notamment permettre de vérifier la disponibilité des postes, utiliser PowerShell Remoting / WinRM pour exécuter des commandes à distance, récupérer les mises à jour installées, rechercher une mise à jour de sécurité identifiée par son numéro KB, détecter les postes conformes ou non conformes, gérer les machines inaccessibles, puis générer un rapport destiné à l'administrateur.

Le projet devra mettre en évidence la chaîne suivante :



L'objectif n'est donc pas seulement d'écrire quelques commandes PowerShell, mais de mettre en œuvre une petite solution centralisée d'audit des correctifs de sécurité d'un parc Windows.



PARTIE 1 — PREPARATION DE L'ENVIRONNEMENT

L'objectif de cette première partie est de mettre en place l'environnement nécessaire à l'administration distante des postes Windows et de vérifier que les différentes machines peuvent communiquer correctement entre elles.

L'environnement de test peut être constitué d'une machine Windows hôte et d'une ou deux machines virtuelles Windows. Il n'est pas nécessaire de disposer d'un grand nombre de machines : l'objectif est de valider le fonctionnement de la solution sur un petit parc, tout en développant un script pouvant être étendu ultérieurement à un nombre plus important de postes.

1. Préparer au minimum deux postes Windows communicants sur le même réseau. Vérifier la connectivité entre les différentes machines à l'aide de commandes PowerShell adaptées.
2. Créer un fichier `computers.txt` contenant le nom et l'adresse IP de chaque machine.
3. Écrire un script PowerShell permettant de lire le contenu de `computers.txt` et d'afficher, pour chaque machine : son nom ; son adresse IP ; son état de disponibilité.

4. Activer et configurer PowerShell Remoting sur les machines concernées. Vérifier l'état du service WinRM sur chaque poste. Tester l'accès distant avec `Test-WSMan`.
5. Établir une première session distante avec : `Enter-PSSession` puis exécuter quelques commandes simples sur la machine distante.
6. Tester ensuite l'exécution distante non interactive avec `Invoke-Command`

A l'issue de cette partie, répondre aux questions suivantes :

- Quel est le rôle de **WinRM** dans PowerShell Remoting ?
- Quelle commande permet d'activer PowerShell Remoting ?
- Quels ports réseau sont utilisés par WinRM ?
- Pourquoi le pare-feu Windows peut-il empêcher une connexion PowerShell distante ?
- Quelle différence existe-t-il entre tester la connectivité avec `Test-Connection` et tester WinRM avec `Test-WSMan` ?
- Pourquoi est-il préférable de vérifier l'accessibilité d'une machine avant de tenter une commande distante ?
- Quels problèmes de sécurité peuvent apparaître si PowerShell Remoting est activé sans restriction ?

PARTIE 2 — INVENTAIRE DU PARC

L'objectif de cette partie est de développer un script PowerShell permettant de recenser automatiquement les postes du parc et de collecter leurs principales caractéristiques système. Le fichier `computers.txt` créé précédemment constituera la source de l'inventaire.

1. À partir du fichier computers.txt, récupérer pour chaque poste :
   - Le nom de la machine ;
   - L'adresse IP ;
   - L'état de la machine : accessible / inaccessible.
2. Pour chaque machine accessible, utiliser PowerShell Remoting afin de collecter les informations suivantes :
   1. Nom du poste ;
   2. Modèle du poste ;
   3. Version du Bios ;
   4. Version et édition de Windows ;
   5. Architecture du système (32/64 bits) ;
   6. Date du dernier démarrage ;
   7. Quantité de mémoire RAM ;
   8. Espace disponible sur le disque système.
3. Compléter l'inventaire en récupérant des informations utiles au Patch Management :
   - Date de la dernière mise à jour installée ;
   - Nombre de mises à jour installées ;
   - État du service Windows Update (wuauserv).

Le script doit continuer son exécution lorsqu'une machine est inaccessible ou lorsqu'une commande distante échoue. L'erreur rencontrée doit être enregistrée dans l'inventaire.

4. Présenter les informations collectées sous la forme d'objets PowerShell puis afficher un tableau récapitulatif, par exemple :

Poste	Adresse IP	Windows	Architecture	Dernier démarrage	Windows Update	État
PC01	192.168.1.20	Windows 11 Pro	64 bits	03/09/2026	Running	Accessible
PC02	192.168.1.21	Windows 10 Pro	64 bits	28/08/2026	Stopped	Accessible
PC03	192.168.1.22	—	—	—	—	Inaccessible

Exporter automatiquement l'inventaire obtenu dans un fichier en .csv.

PARTIE 3 — INVENTAIRE DES CORRECTIFS

L'objectif de cette partie est de développer un script PowerShell permettant de recenser les correctifs Windows installés sur les différents postes du parc et de centraliser les résultats obtenus.

1. À l'aide de PowerShell, affichez sur la machine locale la liste des correctifs installés.
2. Identifiez les informations principales associées à chaque correctif :
   - Numéro de KB ;
   - Description ;
   - Date d'installation ;
   - Utilisateur ou compte ayant effectué l'installation, lorsque l'information est disponible.
3. Utilisez PowerShell Remoting pour récupérer à distance la liste des correctifs installés sur chacune des machines présentes dans `computers.txt`.
4. Le script devra traiter automatiquement l'ensemble des postes accessibles et ignorer proprement les machines inaccessibles, sans interrompre l'exécution globale.
5. Pour chaque poste, construisez un inventaire contenant au minimum :

Poste	Adresse IP	KB	Description	Date d'installation

6. Regroupez l'ensemble des informations collectées dans un fichier `PatchesInventory.csv`
   - Trier les résultats par poste puis par date d'installation décroissante.
   - Afficher pour chaque machine : le nombre total de correctifs recensés ; la date du correctif le plus récent ; l'identifiant KB du dernier correctif installé.

PARTIE 4 — CONTROLE DE CONFORMITE

L'objectif de cette partie est de vérifier automatiquement si les postes du parc respectent une politique minimale de mise à jour définie par l'administrateur. Il ne s'agit plus seulement d'inventorier les correctifs installés, mais de comparer cet inventaire avec une **liste de correctifs obligatoires**.

1. Créez le fichier `required-patches.txt` contenant la liste des mises à jour de sécurité considérées comme obligatoires.
2. Modifiez votre script afin de lire cette liste de correctifs requis. Pour chaque machine accessible du parc, comparez les correctifs installés avec les correctifs obligatoires.
3. Attribuez à chaque poste un état de conformité parmi les suivants : CONFORME, NON CONFORME, INACCESSIBLE. Un poste sera considéré comme conforme uniquement si tous les correctifs présents dans `required-patches.txt` sont installés. Exemple :

Poste	KB requises	KB manquantes	État
PC01	3	0	Conforme
PC02	3	1	Non conforme
PC03	3	—	Inaccessible

4. Pour les machines non conformes, affichez précisément la liste des correctifs manquants. Rajouter aussi la date et l'heure du contrôle.
5. Enregistrez les résultats dans un fichier `ComplianceReport.csv.`
6. Calculez ensuite un taux global de conformité du parc (sans inclure les postes inaccessibles).
7. Pourquoi la politique de conformité doit-elle être régulièrement mise à jour ?

PARTIE 6 — RAPPORT DE SECURITE

L'objectif de cette partie est de centraliser et synthétiser les résultats obtenus lors des différentes phases du projet afin de produire automatiquement un rapport permettant à l'administrateur d'évaluer l'état de conformité du parc et d'identifier rapidement les postes nécessitant une intervention.

1. À partir des informations collectées dans les parties précédentes, générez automatiquement un rapport de sécurité contenant, pour chaque poste :
   - Le nom du poste ;
   - L'adresse IP ;
   - Son état d'accessibilité ;
   - La version de Windows ;
   - La date du dernier démarrage ;

- L'état du service Windows Update ;
- Le nombre de correctifs requis ;
- Le nombre de correctifs manquants ;
- La liste des KB manquantes ;
- L'état de conformité ;
- La date et l'heure du contrôle.

2. Produisez une synthèse globale du parc indiquant au minimum :

```
Nombre total de postes
Nombre de postes accessibles
Nombre de postes inaccessibles
Nombre de postes conformes
Nombre de postes non conformes
Nombre total de correctifs manquants
Taux de conformité
```

2. Ajoutez au rapport une section recensant uniquement les postes nécessitant une intervention
3. Générer une version html du rapport.
4. Le programme devra conserver un historique de ses exécutions dans un fichier de journal, par exemple : PatchManager.log

```
06/09/2026 09:00:01 ; Début de l'audit
06/09/2026 09:00:03 ; PC01 ; Accessible
06/09/2026 09:00:05 ; PC01 ; Conforme
06/09/2026 09:00:07 ; PC02 ; Non conforme ; KBxxxxxxx
06/09/2026 09:00:10 ; PC03 ; WinRM inaccessible
06/09/2026 09:00:11 ; Rapport généré
06/09/2026 09:00:12 ; Fin de l'audit
```

PARTIE 7 — NOTIFICATION

L'objectif de cette partie est de compléter la solution de Patch Management par un mécanisme de notification automatique permettant d'informer l'administrateur lorsqu'une anomalie est détectée. La notification ne devra pas être envoyée systématiquement. Elle devra être déclenchée uniquement lorsqu'au moins une machine présente un problème nécessitant une intervention.

1. À partir du rapport généré dans la partie précédente, détectez automatiquement la présence d'au moins une machine dans l'un des états NON CONFORME ou INACCESSIBLE. Si aucune anomalie n'est détectée, le script devra terminer son exécution sans envoyer de notification. Si une anomalie est détectée, construisez automatiquement un message destiné à l'administrateur :

```
Objet : [PATCH MANAGEMENT] Anomalies détectées

Date du contrôle : 06/09/2026

Postes contrôlés      : 5
Postes non conformes  : 2
Postes inaccessibles  : 1

PC02 : NON CONFORME
       KB503xxxx manquante

PC04 : NON CONFORME
       KB504yyyy manquante

PC05 : INACCESSIBLE
```

Joignez au message le rapport généré précédemment. Évitez d'inscrire directement dans le script des informations sensibles telles qu'un mot de passe de messagerie en clair. Le script devra conserver une trace de l'envoi dans le fichier de journal.

```
06/09/2026 10:15:21 ; Notification envoyée à admin@entreprise.local
```

En cas d'échec :

```
06/09/2026 10:15:21 ; ERREUR ; Notification non envoyée
```

2. Quelles sont les inconvénients de cette solution ?
3. Dans un environnement réel, quelles alternatives modernes peuvent être utilisées à la place d'un simple envoi SMTP depuis le script ?

PARTIE 8 — SECURISATION DE LA SOLUTION

L'objectif de cette partie est d'analyser les risques de sécurité introduits par l'administration distante et de proposer des mesures permettant de protéger la solution de Patch Management. La solution développée devra donc respecter les principes de moindre privilège, contrôle des accès, protection des communications, journalisation et traçabilité.

1. Identifiez les principaux risques liés à l'utilisation de PowerShell Remoting / WinRM dans l'architecture mise en place.
2. Vérifiez quels utilisateurs sont autorisés à ouvrir une session distante sur les postes administrés.
3. Limitez les privilèges du compte utilisé pour le Patch Management au strict nécessaire.
4. Vérifiez que les fichiers produits par la solution ne sont accessibles qu'aux utilisateurs autorisés.
5. Analysez les règles du pare-feu Windows associées à WinRM et vérifiez que l'accès distant n'est autorisé que depuis les machines nécessaires.
6. Vérifiez le mode d'authentification utilisé pour les connexions distantes. Expliquez dans quel cas Kerberos est préférable.
7. Étudiez le principe de Just Enough Administration (JEA) et expliquez comment il pourrait être utilisé pour restreindre les commandes accessibles à distance.
8. Proposez une stratégie de conservation des rapports et des journaux afin d'éviter :
    - Leur modification non autorisée ;
    - Leur suppression ;
    - L'exposition d'informations sensibles sur le parc.

LIVRABLES ET MODALITES DE REMISE

Le projet est à réaliser **individuellement (monôme)** ou en **binôme**.
Date limite de remise : **31 octobre 2026**.
Les livrables suivants sont obligatoires :

Livrable	Contenu attendu
Code source	L'ensemble des scripts PowerShell développés pour le projet, ainsi que les fichiers de configuration et exemples nécessaires à leur exécution.
Dépôt GitHub privé	Le programme devra être déposé dans un projet **GitHub privé**. L'enseignant devra disposer d'un accès au dépôt afin de pouvoir consulter le code source et l'historique du projet.
Rapport	Un rapport synthétique présentant l'architecture de la solution, les fonctionnalités réalisées, les choix techniques, les tests effectués, les résultats obtenus et les mesures de sécurisation mises en œuvre.
Démonstration vidéo	Une vidéo d'environ 15 minutes présentant le fonctionnement de la solution et démontrant son exécution commentée sur l'environnement de test.
Moodle	Le rapport et le lien vers la vidéo de démonstration devront être déposés sur Moodle avant la date limite. Le lien vers le dépôt GitHub privé devra être déposé sur Moodle dès la première séance du projet. Tout lien inaccessible au moment de l'évaluation pourra être considéré comme un livrable non remis.